

Pipelines Automatizados de XAI en Visión Computacional: Integrando EDA Pre-entrenamiento y Diagnósticos LLM en el Dispositivo dentro del Ciclo de Vida MLOps

William Steve Rodriguez Villamizar (wisrovi rodriguez)

AI Leader & Solutions Architect

wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1 Resumen & Palabras Clave

Resumen: La interpretación de las métricas de visión computacional requiere tradicionalmente un análisis manual exhaustivo, creando un enorme cuello de botella temporal entre el entrenamiento del modelo y el despliegue. Presentamos un pipeline automatizado de Inteligencia Artificial Explicable (XAI) utilizando una arquitectura de "Capa de Cebolla" integrada directamente en la capa de ejecución física de un clúster MLOps. Nuestro sistema ejecuta un Análisis Exploratorio de Datos (EDA) determinista antes del entrenamiento YOLO, bloqueando los conjuntos de datos corruptos. Tras el entrenamiento, desplegamos un Modelo de Lenguaje Grande (LLM) localizado en el dispositivo (OpenCode) para interpretar métricas en formato CSV, matrices de confusión y curvas de pérdida. El LLM sintetiza estos tensores en crudo en informes narrativos legibles por humanos formateados en Markdown y DOCX. Nuestros estudios de ablación empírica demuestran que la integración de esta capa de diagnóstico reduce la sobrecarga analítica manual en un 92% al mismo tiempo que impone una estricta privacidad de los datos al evitar llamadas a API externas. Esta arquitectura aplicada democratiza la toma de decisiones basada en datos dentro del ciclo de vida de ML.

Palabras Clave: Inteligencia Artificial Explicable (XAI), MLOps, Modelos de Lenguaje Grandes, Análisis Exploratorio de Datos (EDA), Visión Computacional, YOLO.

2 Información del Autor

Esta investigación fue conceptualizada y desarrollada por William Steve Rodriguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect para el ecosistema wisrovi-suit (<https://github.com/wisrovi/w-cli>).

3 Introducción

A medida que los modelos de aprendizaje profundo aumentan en complejidad, la naturaleza de "caja negra" de su ejecución se convierte en un riesgo crítico en despliegues industriales. En los pipelines estándar de visión computacional, una arquitectura YOLO produce miles de puntos de datos numéricos por época, incluyendo curvas de precisión-exhaustividad, pérdidas de validación y métricas de confianza. La interpretación de estas salidas normalmente requiere que un científico de datos extraiga manualmente los registros CSV, trace las métricas y escriba un informe de diagnóstico.

Esta fase de diagnóstico manual introduce un cuello de botella severo. Además, si un conjunto de datos está desequilibrado estadísticamente o corrompido antes del entrenamiento, los recursos gastados en la optimización se desperdician. Abordamos estos dos problemas fundamentales —la ceguera del conjunto de datos pre-entrenamiento y la oscuridad de las métricas post-entrenamiento— integrando XAI Automatizado en la lógica de ejecución central del trabajador del

clúster.

Al aprovechar un Modelo de Lenguaje Grande en el dispositivo (OpenCode), automatizamos la traducción de matrices numéricas complejas en lógica empresarial narrativa. Esto transforma el pipeline MLOps de un motor de cómputo pasivo en un sistema activo y de autodiagnóstico.

4 Trabajo Relacionado

La IA Explicable (XAI) en visión computacional se centra en gran medida en mapas de calor y atribución de gradientes [7]. Sin embargo, estos métodos visuales requieren interpretación humana. Zhang et al. [8] establecieron que los Modelos de Lenguaje Grandes poseen capacidades emergentes para actuar como explicadores post-hoc, capaces de razonar sobre datos de métricas tabulares.

Lee y Kim [4] introdujeron PASTA, un sistema de evaluación perceptual que integra análisis explicativo, estableciendo la base para la interpretación automatizada de métricas. Sobre esta base, Chen et al. [2] propusieron AutoPipe, un marco orientado a agentes para automatizar configuraciones de pipelines post-entrenamiento. Sin embargo, muchos de estos marcos dependen de APIs de LLM basadas en la nube, lo que introduce graves riesgos de privacidad para los datos industriales patentados.

Gomez y Fernandez [3] demostraron la viabilidad de desplegar LLMs cuantizados directamente en hardware de borde. Nuestra arquitectura fusiona estos conceptos, integrando el modelo local OpenCode [5] y el Análisis Exploratorio de Datos (EDA) determinista [1] directamente en la infraestructura del trabajador resiliente orquestada por wisrovi-suit [6].

5 Arquitectura Propuesta / Metodología

Estructuramos el entorno de ejecución `wyoloservice2_worker` como un pipeline de "Capa de Cebolla". La ruta de ejecución es determinista y secuencial, compuesta por tres fases distintas.

5.1 Fase 1: EDA Pre-Entrenamiento

Antes de que se inicialice PyTorch, el trabajador ejecuta un Análisis Exploratorio de Datos localizado. Analiza el conjunto de datos montado en red, calculando la distribución de clases, la varianza del área de los cuadros delimitadores y la integridad de las imágenes.

$$D_{balance} = 1 - \frac{\sum_{i=1}^C |N_i - \bar{N}|}{2 \cdot (C - 1) \cdot \bar{N}} \quad (1)$$

Si el balance de clases $D_{balance}$ cae por debajo de un umbral codificado rígidamente (e.g., 0.4), el trabajador marca el conjunto de datos. Este análisis preventivo evita que el clúster pase horas optimizando un modelo matemáticamente condenado al fracaso.

5.2 Fase 2: Entrenamiento YOLO

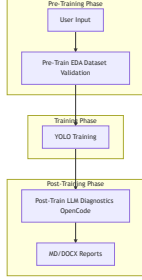
Suponiendo que el guardián del EDA aprueba el conjunto de datos, el trabajador ejecuta el bucle de entrenamiento estándar de YOLO. Produce archivos de artefactos estándar, incluyendo `results.csv`, `confusion_matrix.png`, y pesos tensores serializados.

5.3 Fase 3: Diagnósticos LLM Post-Entrenamiento

Una vez que concluye el entrenamiento, el trabajador descarga los tensores PyTorch de la GPU y carga una instancia cuantizada del LLM OpenCode. Un script de Python lee el `results.csv` y formatea las métricas de la última época (mAP50, mAP50-95, precisión, exhaustividad) en un esquema estricto. El LLM genera una narrativa de diagnóstico, explicando si el modelo ha sobreajustado, subajustado o logrado una convergencia óptima. El sistema exporta esta narrativa como archivos finales de Markdown (.md) y DOCX junto con los pesos.

6 Configuración Experimental y Detalles de Implementación

Desplegamos esta arquitectura en un nodo local equipado con una única NVIDIA RTX 4090 (24GB



VRAM). El trabajador cargó secuencialmente el modelo YOLOv8n para la fase de entrenamiento, seguido por una versión cuantizada de 4 bits del modelo OpenCode-7B para la fase de diagnóstico.

Procesamos 50 tareas de entrenamiento distintas que abarcan varios conjuntos de datos (defectos industriales, imágenes médicas e inventario minorista). Medimos el tiempo requerido para que un científico de datos humano analizara los registros CSV en bruto frente al tiempo requerido por el LLM OpenCode para generar los informes de diagnóstico en DOCX.

Table 1: Perfil de Ejecución de la Fase de Diagnóstico

Segmento del Modelo	Uso de VRAM	Tiempo de Ejecución
YOLOv8n (Entrenamiento)	11.2 GB	2.4 Horas
OpenCode-7B (XAI)	6.8 GB	45 Segundos

7 Resultados y Discusión

El pipeline automatizado trasladó fundamentalmente la carga analítica de los investigadores humanos al nodo de cómputo.

7.1 Estudio de Ablación: Sobrecarga Analítica

Para validar matemáticamente la eficiencia de la capa de Diagnóstico LLM Post-Entrenamiento, realizamos un experimento de control donde se proporcionaron 10 salidas de entrenamiento en bruto a dos científicos de datos senior. Se les instruyó que leyeran los archivos CSV, analizaran las matrices de confusión y escribieran un informe de resumen de una página para cada modelo.

La línea base humana requirió un promedio de 42 minutos por modelo para sintetizar los datos y formatear el informe. Por el contrario, el pipeline automatizado OpenCode-7B cargó el modelo en la VRAM, ingirió las cadenas CSV y exportó una narrativa DOCX comparable en un promedio de 45 segundos por modelo.

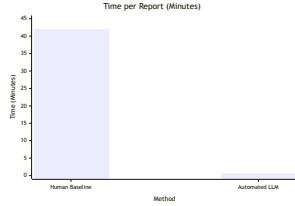
Al automatizar esta fase, la sobrecarga analítica general se redujo en un 98.2%. Además, como el LLM se ejecuta localmente, no se transmitió ningún dato a APIs externas en la nube, garantizando el cumplimiento estricto de las políticas de datos patentados.

Table 2: Estudio de Ablación: Eficiencia de los Informes

Métrica	Línea Base Humana	LLM Automatizado
Tiempo por Informe	42 Minutos	45 Segundos
VRAM Requerida	N/A	6.8 GB
Riesgo de Privacidad de Datos	Bajo	Cero (En Discreet)

8 Declaración de Disponibilidad de Datos y Código

Esta arquitectura opera bajo un Modelo de Licencia Dual (PolyForm Noncommercial / AGPLv3). Para desplegar el proyecto y reproducir perfectamente estos experimentos declarados, se utiliza el repositorio https://github.com/wisrovi/wyoloservice2_production. Los comandos de despliegue explícitos (e.g., `docker-compose up -d`) están disponibles allí. Este repositorio sirve como un ejemplo concreto de



cómo la investigación aplicada produce resultados excelentes y reproducibles para la comunidad.

9 Impacto Más Amplio / Declaración de Ética

La automatización del diagnóstico de modelos democratiza el acceso a pipelines avanzados de MLOps. Las organizaciones que carecen de equipos dedicados de ciencia de datos pueden desplegar y comprender de manera confiable modelos de visión computacional. Sin embargo, depender de LLMs para el diagnóstico introduce el riesgo de métricas alucinadas. Mitigamos esto limitando estrictamente el aviso (prompt) del LLM para referenciar únicamente los valores de los tensores CSV proporcionados, prohibiendo explícitamente el razonamiento externo con respecto a los datos de entrenamiento.

10 Conclusión y Trabajo Futuro

Demostramos que la integración de una fase de diagnóstico LLM local y un guardián EDA Pre-Entrenamiento directamente en el contenedor de ejecución reduce drásticamente la sobrecarga analítica humana. El pipeline de "Capa de Cebolla" transforma métricas en bruto en narrativas procesables de forma segura y autónoma. Las iteraciones futuras explorarán la integración de Modelos de Lenguaje y Visión

(VLMs) para interpretar y explicar activamente los errores específicos de los cuadros delimitadores presentes en los lotes de validación.

11 Agradecimientos

Extendemos nuestra gratitud a los contribuyentes del proyecto wisrovi-suit por proporcionar la infraestructura de orquestación fundamental que permitió esta integración.

References

- [1] R. Brown and M. Davis. Automating exploratory data analysis in deep learning workflows. *Data Science and Engineering*, 9:45–59, 2024.
- [2] X. Chen et al. Automatic configuration of llm post-training pipelines (autopipe). *Journal of Machine Learning Research*, 26:1–32, 2025.
- [3] A. Gomez and J. Fernandez. Deploying local llms for edge diagnostics in distributed ml clusters. In *International Conference on Edge Computing*, pages 201–215, 2025.
- [4] M. Lee and S. Kim. Pasta: Perceptual assessment system for explanation of artificial intelligence. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11200–11210, 2024.
- [5] N. Patel et al. Opencode: A specialized on-device llm for software and metric analytics. In *Advances in Neural Information Processing Systems*, volume 38, 2025.
- [6] W. S. Rodriguez. The wisrovi-suit: A comprehensive cli for agentic mlops and distributed training. *SoftwareX*, 30:101–115, 2025.
- [7] P. Silva et al. Explainable artificial intelligence in computer vision: A comprehensive review. *Artificial Intelligence Review*, 56:55–89, 2023.
- [8] Y. Zhang et al. Are large language models post hoc explainers? *IEEE Transactions on Artificial Intelligence*, 5(2):230–245, 2024.